

Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI

Email: vtu29748@veltech.edu.in

Roll no: vtu29748

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 2_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Manoj loves puzzles and recently encountered an interesting mathematical challenge. He needs to find how many ways he can express a number x as the sum of distinct positive integers raised to the power of n . For example, for a given number x , he wants to find all possible ways to represent it as a sum of distinct powers of integers.

Help him implement this task.

Example 1

Input:

10

2

Output:

1

Explanation:

$x = 10$

$n = 2$

$10 = 12 + 32$, hence we have only 1 possibility.

Example 2

Input:

100

2

Output:

3

Explanation:

$x = 100$

$n = 2$

$100 = 102$ OR $62 + 82$ OR $12 + 32 + 42 + 52 + 72$, hence total 3 possibilities.

Function Specifications: `getAllWays(int, int, int)`

Input Format

The first line of input consists of an integer x , representing the target sum.

The second line of input consists of an integer n , representing the power to which the integers are raised.

Output Format

The first line of output prints: The total number of ways to express x as the sum of distinct powers.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 100

2

Output: 3

Answer

```
#include<stdio.h>
#include<math.h>
int getAllWaysUtil(int x,int n,int num){
    int power=(int)pow(num,n);
    if(power>x){
        return 0;
    }
    else if(power==x){
        return 1;
    }
    else{
        return getAllWaysUtil(x-power,n,num+1)+getAllWaysUtil(x,n,num+1);
    }
}
int getAllWays(int x,int n){
    return getAllWaysUtil(x,n,1);
}
int main(){
    int x,n;
    scanf("%d",&x);
    scanf("%d",&n);
    int result=getAllWays(x,n);
    printf("%d\n",result);
    return 0;
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an integer array of coins of size N representing different types of

currency and an integer sum M. The task is to find the number of ways to make the sum by using different combinations from the coins array, using recursion.

Function Specifications: `int countWays(int [], int , int)`

Input Format

The first line of input consists of the number of coins N.

The second line consists of N integers denoting the coin denominations.

The third line consists of the integer sum M to be produced with those coins.

Output Format

The output displays the number of ways to make the sum by using different combinations from the coins array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

1 2 3

4

Output: 4

Answer

```
#include <stdio.h>
```

```
int countWays(int coins[], int n, int sum) {  
    if (sum == 0)  
        return 1;  
    if (sum < 0)  
        return 0;  
    if (n <= 0 && sum > 0)  
        return 0;  
    return countWays(coins, n - 1, sum) + countWays(coins, n, sum - coins[n - 1]);  
}
```

```
int main() {
    int n, sum;
    scanf("%d", &n);
    int coins[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &coins[i]);
    scanf("%d", &sum);
    printf("%d\n", countWays(coins, n, sum));
    return 0;
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Meredith is a cashier at a local convenience store. To improve the efficiency of making change for customers, Meredith wants to develop a small program that calculates the number of different ways she can give change using a given set of coin denominations. This will help her quickly figure out the number of ways to make change for a particular amount of money using the available coins.

Meredith has a list of coin denominations and a specific amount of money for which she needs to provide change. She wants to write a program to determine the number of possible ways to achieve the given amount using the available denominations. This will help her understand all possible combinations without having to manually calculate them each time.

Input Format

The first line contains an integer n , the number of different coin denominations.

The second line contains n space-separated integers representing the coin denominations.

The third line contains an integer m , the target amount of money for which Meredith needs to make change.

Output Format

The output displays the number of ways to make the sum by using different

combinations from the coins array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

1 2 3

4

Output: 4

Answer

```
#include <stdio.h>
```

```
int countWays(int coins[], int n, int sum) {  
    if (sum == 0)  
        return 1;  
    if (sum < 0)  
        return 0;  
    if (n <= 0 && sum > 0)  
        return 0;  
    return countWays(coins, n - 1, sum) + countWays(coins, n, sum - coins[n - 1]);  
}
```

```
int main() {  
    int n, sum;  
    scanf("%d", &n);  
    int coins[n];  
    for (int i = 0; i < n; i++)  
        scanf("%d", &coins[i]);  
    scanf("%d", &sum);  
    printf("%d\n", countWays(coins, n, sum));  
    return 0;  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

In a small research lab, a programmer named Arin is analyzing binary data to detect system flags stored as set bits in an integer. One day Arin receives an integer and must determine how many bits are set to 1 in its binary representation.

Write a program to ensure that this task is done so Arin can continue validating the system's binary logs. The objective is to count all set bits in the given number using any valid approach.

Function specification: countSetBits(int n)

Input Format

The input consists of a single positive integer n.

Output Format

The output prints a single integer representing the count of set bits (1s) in the binary representation of n.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 21

Output: 3

Answer

```
#include <stdio.h>
```

```
int countSetBits(int n) {  
    int count = 0;  
    while (n) {  
        count += n & 1;  
        n >>= 1;  
    }  
    return count;  
}
```

```
int main() {
```

```
int n;  
scanf("%d", &n);  
printf("%d\n", countSetBits(n));  
return 0;  
}
```

Status : Correct

Marks : 10/10

5. Problem Statement

Raju is a computer graphics designer. He works around matrices with ease. His friend challenged him to rotate an $M \times M$ matrix clockwise with an angle of 90 degrees.

However, Raju was given a constraint that he must not use any additional 2D matrix for doing the above-mentioned task. The rotation must be done in place, which means Raju has to modify the given input matrix directly.

Example: 1

Input:

[[1,2,3],[4,5,6],[7,8,9]]

Output:

[[7,4,1],[8,5,2],[9,6,3]]

Example: 2

Input:

matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]

Output:

[[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

Input Format

The first line of the input is a single integer value for the size of the row and column, M.

The next M lines of the input are the matrix elements, each containing M integers representing the elements of the matrix.

Output Format

The output displays the rotated matrix, with each row printed on a new line.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 3

1 2 3

4 5 6

7 8 9

Output: 7 4 1

8 5 2

9 6 3

Answer

```
#include <stdio.h>
```

```
void rotateMatrix(int matrix[][1000], int n) {
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = i; j < n; j++) {
```

```
            int temp = matrix[i][j];
```

```
            matrix[i][j] = matrix[j][i];
```

```
            matrix[j][i] = temp;
```

```
        }
```

```
    }
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0, k = n - 1; j < k; j++, k--) {
```

```
            int temp = matrix[i][j];
```

```
            matrix[i][j] = matrix[i][k];
```

```
            matrix[i][k] = temp;
```

```
        }
```

```
    }
```

```
}
```

```
int main() {
    int n;
    scanf("%d", &n);
    int matrix[1000][1000];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }
    rotateMatrix(matrix, n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Status : Correct

Marks : 10/10